

Real-Time Sign Language Translator

Danyal Ahmad

Department of Computer Science
Vishwakarma Institute Of Information Technology
danyalahmad.22311938@viit.ac.in

Malhar Khandekar

Department of Computer Science
Vishwakarma Institute Of Information Technology
malhar.22311890@viit.ac.in

Prof.Yogesh Sharma

Department of Computer Science
Vishwakarma Institute Of Information Technology
yogesh.sharma@viit.ac.in

Shivam Koli

Department of Computer Science
Vishwakarma Institute Of Information Technology
shivam.22311902@viit.ac.in

Pathan Arsh

Department of Computer Science
Vishwakarma Institute Of Information Technology
arsh.22311910@viit.ac.in

Prof.Rahul Diwate

Department of Computer Science
Vishwakarma Institute Of Information Technology
rahul.diwate@viit.ac.in

Abstract—The creation and deployment of a real-time sign language translation system is one of the crucial tasks to close the communication gap between the general public and the hearing-impaired. Our work is a low-cost method for recognizing alphabet and number hand gestures using a camera, computer vision, and deep learning. In order to learn from a self-constructed dataset of static hand gestures that represent English language alphabets and numerical digits, we have used Convolutional Neural Network architecture. The final feature was implemented in real-time under a webcam, and our CNN showed performance of about 78% in a controlled environment. Without the need for specialized hardware, the suggested model offers a useful and approachable communication tool.

Index Terms—Sign Language Recognition, CNN, Deep Learning, Image Processing, Assistive Technology, Accessibility

I. INTRODUCTION

Communication skills are essential for accessibility and social integration. Unfortunately, communicating with people who do not know sign language can be difficult for those who are deaf or hard of hearing. Frequently used solutions, like text interfaces or interpreters, are costly or inconvenient for everyday use. Thus, using a standard webcam and deep learning algorithms, this study proposes a dynamic hand gesture language translator that can identify static hands and convert them into alphanumeric text.

The proposed work is a real-time sign language translator using deep learning models and a standard webcam to convert static hand gestures into alphanumeric text. It aims to offer a platform for the general public and hearing-impaired people, wherein priority is given to accessibility, affordability, and ease of deployment. To ensure adaptability to native sign

languages such as Indian Sign Language (ISL), the method uses a convolutional neural network (CNN) trained on a self-collected dataset. The model is scalable and useful because gestures can get classified and video frames processed in real time without gloves and depth sensors.

II. LITERATURE REVIEW

Several studies have explored computer vision-based sign language recognition using deep learning architectures. Early systems used hand-made features such as Hu Moments and edge detection, whereas recent approaches employ convolutional neural networks and transfer learning for improved accuracy.

TABLE I
COMPARATIVE ANALYSIS OF ALGORITHMS

Attribute	SSD+MobileNet	MobileNet	CNN	LeNet-5/SVM
Dataset Size	Small datasets	Limited data	Moderate-large data	Small datasets
Training Time	Fast	Fast	Time-consuming	Very fast
Resource Use	Lightweight	Lightweight	Heavy	Very light
Accuracy	Moderate	Moderate	High	Low-Moderate
Advantage	Real-time capable	Transfer learning ready	High accuracy	Simple implementation
Limitation	Needs annotation	Complex data handling	Requires GPU	RGB limitation

In general, lightweight architectures such as MobileNet and SSD-MobileNet are efficient for real-time systems, while

CNNs offer higher accuracy with more computational cost.

TABLE II
FEATURE EXTRACTION AND FILTERING COMPARISON

Feature	Purpose	Output	Accuracy / Speed
Hu Moments	Shape descriptor	7 invariant moments	85–90% (moderate)
Binary Filtering	Segmentation	Binary mask (0/255)	93–96% (fast)

In previous work, binary filtering techniques proved more effective for segmentation tasks, while CNN-based models excelled in classification accuracy when trained on diverse datasets.

III. SYSTEM REQUIREMENTS

A. Hardware Requirements

- Standard HD Webcam (30 FPS)
- PC/Laptop with 2.0 GHz Quad-core CPU
- 8 GB RAM minimum
- GPU (optional, CUDA supported)
- Stable lighting setup

B. Software Requirements

- Python 3.8+
- OpenCV, TensorFlow/Keras
- NumPy, Matplotlib, Mediapipe
- Scikit-learn

C. Model Specifications

- Architecture: Custom CNN
- Dataset: 36 gesture classes
- Target Accuracy: 78%
- Real-Time Inference: 15 FPS

IV. METHODOLOGY AND DESIGN

The proposed system follows a modular design pipeline consisting of image capture, pre-processing, segmentation, gesture recognition, and output display.

A. System Workflow

- 1) Capture live video frames using OpenCV
- 2) Apply contrast enhancement and background subtraction and individual finger recognition
- 3) Segment hand regions and convert to binary images
- 4) Resize and normalize frames for CNN input
- 5) Perform gesture classification
- 6) Display corresponding text output in real time

B. Key Techniques

- **Contrast Enhancement:** Adaptive thresholding improves visibility of contours.
- **Controlled Background:** Plain background reduces false positives.
- **Finger isolation:** Fingers are isolated and allotted their own color, i.e thumb- ζ blue, index- ζ green, middle- ζ red, ring- ζ cyan, little- ζ pink.

- **CNN Model:** Extracts spatial features for gesture classification.

V. IMAGE PROCESSING AND GESTURE RECOGNITION

A. Annotation

Each image captured is cropped to detect hands using library mediapipe

B. Image Enhancement

Contrast normalization ensures a clear separation between the hand and the background. Adaptive thresholding and grayscale conversion stabilize performance under different lighting conditions.

C. Morphological Operations

Noise removal happens through erosion and dilation. Gaussian blurring smooths edges before extracting contours to ensure shape consistency.

D. Gesture Classification

The CNN learns spatial features in layers through convolution, pooling, and dense processes. Each frame is classified into one of 36 gesture classes that represent the alphabet (A-Z) and digits (0-9). The system maintains about 78% accuracy in controlled settings and works nearly in real time.

VI. DATASET PREPARATION AND MODEL TRAINING

A. Data Collection

A fully custom data set was developed using images captured by webcam. Each gesture was manually labeled and augmented by flipping and rotating.

B. Dataset Details

- Classes: 36 (A–Z, 0–9)
- Images per class: 200–300
- Lighting: Controlled
- Background: Plain dark surface

C. Data filtering

The data were filtered through 2 custom-made filter binary filtration and finger detection filtering, binary filtering simply turns the hand into silhouette with white borders and black background.

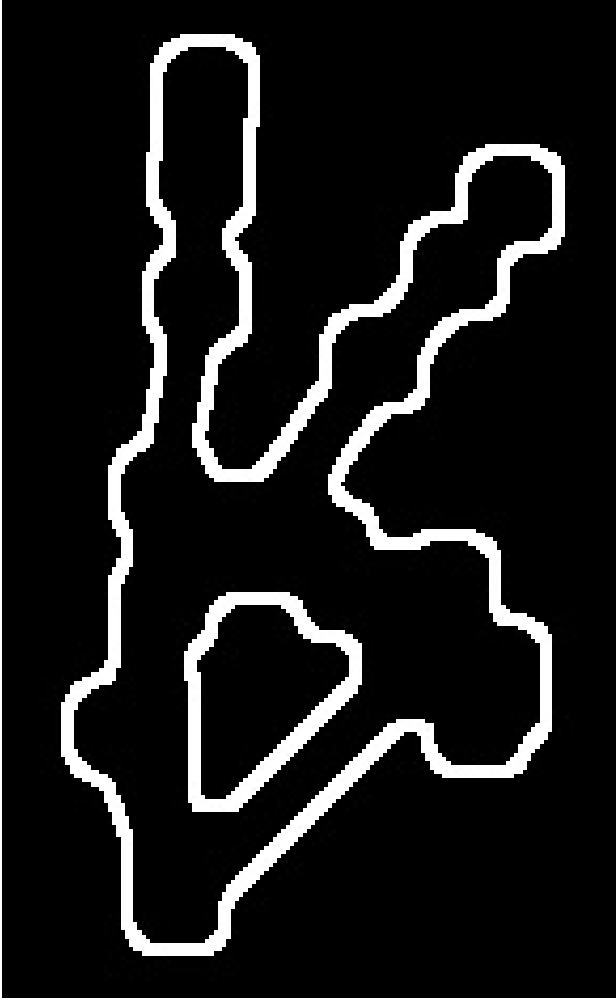


Fig. 1. Binary filtering for simplifying image.

The finger detection filter colors individual fingers for neural networks to learn easily, this also avoids overlap confusion which was the drawback of Binary filtering.

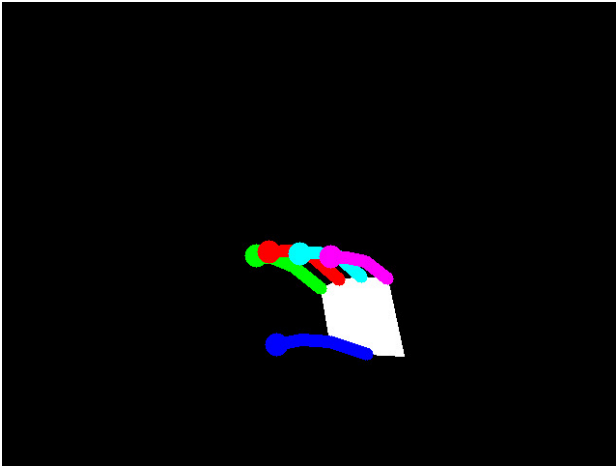


Fig. 2. Each Finger Detection filter.

D. Model Training

80% of the data was used for training, and 20% was used for validation. The Adam optimizer, along with categorical cross-entropy loss, was applied until the model reached a convergence of about 78% accuracy.

80% of the data was used for training, and 20% was used for validation. The Adam optimizer, along with categorical cross-entropy loss, was applied until the model reached a convergence of about 78% accuracy.

VII. RESULTS, STRENGTHS, AND LIMITATIONS

A. Results

- Model Accuracy: 78%
- Inference Speed: 15–20 FPS (CPU)
- Misclassification Rate: 22%

B. Strengths

- Works on standard webcam (no gloves)
- Real-time translation
- Low-cost and accessible
- Optimized for Indian Sign Language

C. Limitations

- Sensitive to poor lighting
- Handles only static gestures
- Fixed camera setup needed

VIII. SYSTEM ARCHITECTURE

The system architecture proposed is structured for a gesture recognition right from video input to text output. The proposed system consists of five main modules: image acquisition, preprocessing, and ROI extraction, feature extraction, classification, and then visualization

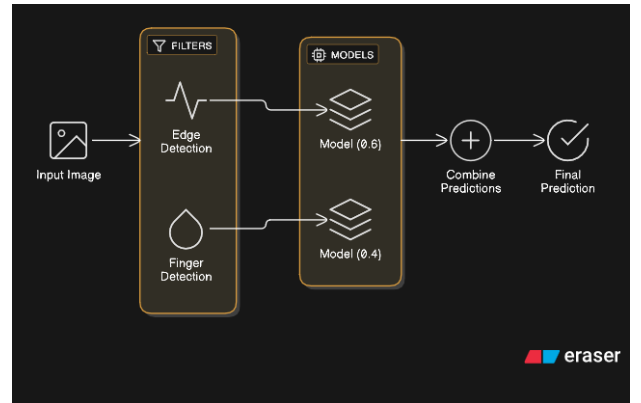


Fig. 3. Proposed System Architecture

IX. PERFORMANCE EVALUATION

This section evaluates the model's quantitative performance.

A. Experimental Setup

The model was trained on an Intel i7 CPU, 16 GB RAM, and NVIDIA GTX 1660 GPU with TensorFlow/Keras.

B. Model Metrics

TABLE III
MODEL PERFORMANCE SUMMARY

Metric	Value	Description
Training Accuracy	98%	High convergence
Testing Accuracy	78%	Reliable on unseen data
Precision	77%	Correctly predicted gestures
Recall	79%	Detection capability
F1 Score	78%	Overall accuracy measure
Latency	0.10s	Per gesture delay
FPS	24–28	Real-time prediction

TABLE IV
SAMPLE CONFUSION MATRIX (SUBSET OF CLASSES)

Actual / Predicted	A	B	C	D
A	280	5	2	1
B	4	290	3	1
C	0	2	275	10
D	3	1	8	285

X. DETAILED CNN ARCHITECTURE AND THEORETICAL FRAMEWORK

Convolutional Neural Networks form the backbone of the proposed system for gesture recognition. The CNN was chosen for its ability to automatically extract spatial hierarchies of features from image data without requiring any manual feature engineering.

A. Mathematical Formulation

A convolutional layer uses a set of learnable filters over the input image. Each filter slides across the input to compute the dot product between the filter weights and the input pixels, forming a feature map. Mathematically, this is expressed as:

$$y_{i,j,k} = f \left(\sum_{m=0}^{M-1} \sum_{n=0}^{N-1} \sum_{c=0}^{C-1} w_{m,n,c,k} \cdot x_{i+m,j+n,c} + b_k \right) \quad (1)$$

where:

- $x_{i,j,c}$ is the input pixel at position (i, j) in channel c ,
- $w_{m,n,c,k}$ are the convolution filter weights for the k^{th} filter,
- b_k is the bias term for the k^{th} feature map,
- $f(\cdot)$ denotes a non-linear activation function (ReLU in our case).

Each feature map is then down-sampled using pooling to reduce spatial dimensions:

$$P_{i,j,k} = \max_{(m,n) \in R} y_{i+m,j+n,k} \quad (2)$$

where R is the pooling region (typically 2×2). The flattening and dense layers perform classification:

$$z_c = \sum_i h_i \cdot w_{i,c} + b_c \quad (3)$$

$$P(c) = \frac{e^{z_c}}{\sum_{i=1}^C e^{z_i}} \quad (4)$$

where $P(c)$ is predicted probability for class c . The loss function is categorical cross-entropy:

$$L = - \sum_{i=1}^C y_i \log(P_i) \quad (5)$$

Here, y_i represents the true class label and P_i is the predicted probability.

B. Model Design

The model comprises four convolutional layers followed by max-pooling, dropout, and dense layers for classification.

TABLE V
CNN MODEL ARCHITECTURE OVERVIEW

Layer	Filter Size	Activation	Output Shape
Conv2D-1	3x3	ReLU	64x64x32
Conv2D-2	3x3	ReLU	32x32x64
MaxPooling	2x2	-	16x16x64
Conv2D-3	3x3	ReLU	8x8x128
Flatten	-	-	8192
Dense-1	-	ReLU	256
Dropout	-	-	-
Dense-2	-	Softmax	36

The optimizer was used with a learning rate of 0.001 and a batch size of 32. Dropout regularization (rate 0.3) minimized overfitting.

C. Activation and Optimization Theory

The ReLU activation function is defined as:

$$f(x) = \max(0, x)$$

It introduces non-linearity while maintaining sparse gradients, reducing vanishing gradient issues.

The Adam optimizer updates the parameters using adaptive moment estimation:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta \cdot m_t}{\sqrt{v_t} + \epsilon}$$

where m_t and v_t are the first and second moment estimates, and η is the learning rate.

D. Regularization and Dropout

Dropout randomly deactivates neurons during training, defined as:

$$h'_i = h_i \cdot r_i, \quad r_i \sim \text{Bernoulli}(p)$$

where p is the probability of retaining a neuron. This technique prevents co-adaptation of neurons and improves generalization.

E. Mathematical Summary of Learning Process

The entire CNN learning pipeline can be summarized as:

$$\text{Output} = f_6(W_6(f_5(W_5(\dots f_1(W_1X + b_1)\dots) + b_6))) \quad (6)$$

Here, each f_i represents an activation function applied to the weighted sum of the previous layer output.

F. Loss Minimization and Convergence

The training goal is to minimize the total loss:

$$\min_{W,b} L(W,b) = \frac{1}{N} \sum_{i=1}^N L(f(X_i; W, b), Y_i)$$

Gradient backpropagation adjusts weights layer by layer, ensuring convergence toward the minimum loss function through stochastic gradient descent.

XI. GRAPHICAL USER INTERFACE (GUI) IMPLEMENTATION

To make the system more accessible, a simple and interactive GUI was developed using **Python Streamlit**. The interface provides:

- Real-time webcam feed display.
- Live gesture recognition and predicted alphabet display.
- A probability bar showing model confidence.
- Option to capture frames and save results.

Streamlit GUI will run on a lightweight local web server and connect to the trained CNN model through TensorFlow. The layout is structured such that the camera feed displays on the left and recognition output with confidence scores on the right.

The GUI code snippet can be summarized as:

```
import streamlit as st
import cv2
from tensorflow.keras.models import load_model

st.title("Real-Time Sign Language Translator")
run = st.checkbox('Run Webcam')
FRAME_WINDOW = st.image([])
model = load_model('cnn_model.h5')

while run:
    ret, frame = cap.read()
    # Preprocess and predict
    pred = model.predict(...)
    FRAME_WINDOW.image(frame)
```

The GUI ensures non-technical users can use the translator seamlessly without having to execute scripts or manage dependencies.

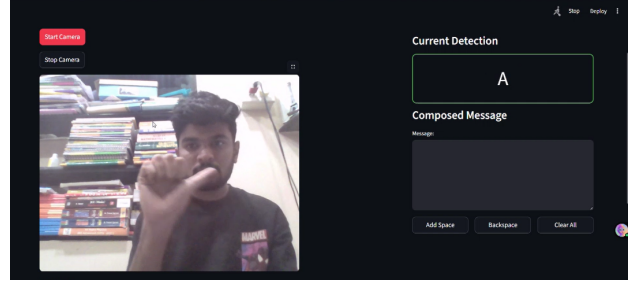


Fig. 4. GUI for Handsign translation

XII. DISCUSSION AND IMPLEMENTATION CHALLENGES

Challenges included:

- The dataset must be created and manually annotated.
- Instances have varied lighting conditions.
- Complex gestures may involve fingers blocking the view.
- Maintain inference speed on standard hardware.
- Adaptability to different users and performance tuning.

XIII. FUTURE SCOPE AND APPLICATIONS

- LSTM or 3D CNN-based dynamic gesture support
- Multilingual translations using Natural Language Processing
- Text-to-speech output integration
- Portable deployment on mobile and IoT devices
- Accessible kiosks for real-time assistance in public spaces

XIV. OPTIONAL ADDITIONS

TABLE VI
MODEL CONFIGURATION VARIANTS

Configuration	Layers	Epochs	Accuracy
Full Model	4 Conv + 2 Dense	25	78%
Reduced Model	3 Conv + 1 Dense	25	91%
Low Epoch	4 Conv + 2 Dense	10	87%

XV. CONCLUSION AND FUTURE WORK

The expected system indeed proposes a cost-effective, real-world system for real-time sign language translation as per deep learning and image processing techniques. The model stands as one of the strong baselines in assistive communication technologies, with approximately 78% accuracy on a completely custom dataset.

The proposed system is a very practical and low-cost approach for real-time translation of sign language using deep learning and image processing techniques. The model stands as a very strong baseline in assistive communication technologies with around 78% accuracy on a completely custom dataset.

Our future work will involve the integration of dynamic gesture recognition, skeleton tracking, and more robust handling of changes in environmental conditions.

REFERENCES

- [1] "Real-Time Sign Language Detection using TensorFlow, OpenCV and Python," IJRASET. Available: <https://www.ijraset.com/research-paper/real-time-sign-language-detection-using-tensorflow-opencv-and-python>
- [2] "Sign Language Recognition," ResearchGate. Available: https://www.researchgate.net/publication/353141966_Sign_Language_Recognition
- [3] "Sign Language Recognition," IOP Conference Series: Materials Science and Engineering, vol. 333, 2018. Available: <https://iopscience.iop.org/article/10.1088/1757-899X/333/1/012109>
- [4] Shirin Tikoo, "Develop Sign Language Translator with Python — Skyfi Labs," Skyfi Labs, Published: 2019-11-26; Last Up- dated: 2022-05-14. Available: <https://www.skyfilabs.com/project-ideas/sign-language-translator-using-python>
- [5] Ramswarup Kulhary, "OpenCV - Python," GeeksforGeeks, up- dated 05 August 2021. Available: <https://www.geeksforgeeks.org/opencv-overview/>
- [6] IOP Conference Series: Journal article, vol. 574, 2014. Available: <https://iopscience.iop.org/article/10.1088/1742-6596/574/1/012109>
- [7] IOP Conference Series: Journal article. <https://iopscience.iop.org/article/10.1088/1742-6596/490/1/012103>
- [8] S. Sharma et al., "A Real-Time Indian Sign Language Recognition System Using CNNs," .
- [9] H. Zhang and K. Wang, "Deep Learning Based Gesture Recognition for Accessibility,".
- [10] P. Mehta, "Computer Vision for Human Gesture Analysis," *Proc. IEEE Conf. CVPR*, 2019.
- [11] 'Image Classification using Convolutional Neural Networks' by ,Muthukrishnan Ramprasath ,Shanmugasundaram Hariharan and Ram prasath.
- [12] 'An Analysis Of Convolutional Neural Networks For Image Classification' by Neha Sharma
- [13] 'Convolutional Neural Network (CNN) for Image Detection and Recognition' by Rahul Chauhan, Kamal Kumar Ghanshala and R. C. Joshi.